

ApexPlanet Cybersecurity Internship

Task 3 Report

Name: Vadla Laxmi

Task: Web Application Security

Internship Id: APSPL2633554

Date: 26-06-2026

Abstract

This report presents the implementation of **Task 3 – Web Application Security** as part of the **ApexPlanet Cybersecurity Internship Program**. The primary objective of this task was to understand, identify, and analyze common web application vulnerabilities in a controlled laboratory environment using **Kali Linux** and **Damn Vulnerable Web Application (DVWA)**. The practical exercises focused on several **OWASP Top 10** vulnerabilities, including **SQL Injection (SQLi)**, **Cross-Site Scripting (Stored and Reflected XSS)**, **Cross-Site Request Forgery (CSRF)**, and **Local File Inclusion (LFI)**. Additionally, **Burp Suite Community Edition** was used to intercept and modify HTTP requests, perform request analysis, and understand web application communication. HTTP security headers were also examined and configured to strengthen browser-side security.

Each vulnerability was demonstrated in a safe and isolated environment to understand its working mechanism, potential impact, and associated security risks. Appropriate mitigation techniques, such as **prepared statements**, **input validation**, **output encoding**, **Content Security Policy (CSP)**, **CSRF tokens**, **secure file handling**, and **HTTP security headers**, were studied and documented. The activities performed during this task enhanced practical knowledge of web application penetration testing, secure coding practices, and defensive security mechanisms.

This project provided valuable hands-on experience in ethical hacking methodologies and emphasized the importance of identifying and mitigating security vulnerabilities during the software development lifecycle. All experiments were conducted exclusively in a controlled laboratory environment for educational purposes, ensuring responsible and ethical cybersecurity practices.

Objectives

The primary objectives of this project are:

- To understand the fundamentals of **Web Application Security** and the **OWASP Top 10** vulnerabilities.
- To set up a secure web application testing environment using **Kali Linux** and **DVWA (Damn Vulnerable Web Application)**.
- To identify and demonstrate common web application vulnerabilities such as **SQL Injection (SQLi)**, **Cross-Site Scripting (Stored and Reflected XSS)**, **Cross-Site Request Forgery (CSRF)**, and **Local File Inclusion (LFI)**.
- To use **Burp Suite Community Edition** for intercepting, analyzing, and modifying HTTP requests and responses.
- To understand the impact of insecure coding practices and how attackers exploit web application vulnerabilities.
- To implement and study mitigation techniques, including **prepared statements**, **input validation**, **output encoding**, **Content Security Policy (CSP)**, **CSRF protection mechanisms**, and **secure file handling**.
- To analyze and improve **HTTP Security Headers** for enhancing browser-side security.
- To document the complete testing methodology, findings, screenshots, and mitigation strategies in a professional report.
- To develop practical skills in ethical hacking and secure web application testing within a controlled laboratory environment.
- To strengthen knowledge of secure coding practices and responsible cybersecurity principles for real-world applications.

Hardware Requirements

The following hardware components were required to set up the web application security testing environment and successfully perform the experiments in this project:

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended for better performance)
- **Storage:** At least 40 GB of free disk space
- **System Architecture:** 64-bit (x64) processor
- **Virtualization Support:** Intel VT-x or AMD-V enabled in BIOS/UEFI

- **Internet Connection:** Required for downloading software packages, updates, and DVWA
- **Input Devices:** Standard keyboard and mouse
- **Display:** Minimum screen resolution of 1366 × 768

These hardware specifications provide sufficient resources to run a virtual machine with Kali Linux, Apache2, MariaDB, DVWA, Burp Suite, and other tools required for performing web application security testing in a controlled laboratory environment.

Software Requirements

The following software was used to set up the web application security testing environment and perform all the experiments included in this project:

- **Operating System:** Kali Linux
- **Virtualization Software:** Oracle VM VirtualBox
- **Web Server:** Apache2
- **Database Server:** MariaDB
- **Server-Side Scripting Language:** PHP
- **Vulnerable Web Application:** Damn Vulnerable Web Application (DVWA)
- **Web Browser:** Mozilla Firefox
- **Web Application Testing Tool:** Burp Suite Community Edition
- **Version Control System:** Git
- **Command-Line Interface:** Kali Linux Terminal
- **HTTP Header Testing Tool:** curl
- **Text Editor:** Nano

These software components were installed and configured to create a secure and isolated laboratory environment for performing web application security testing, vulnerability assessment, request interception, and implementing mitigation techniques.

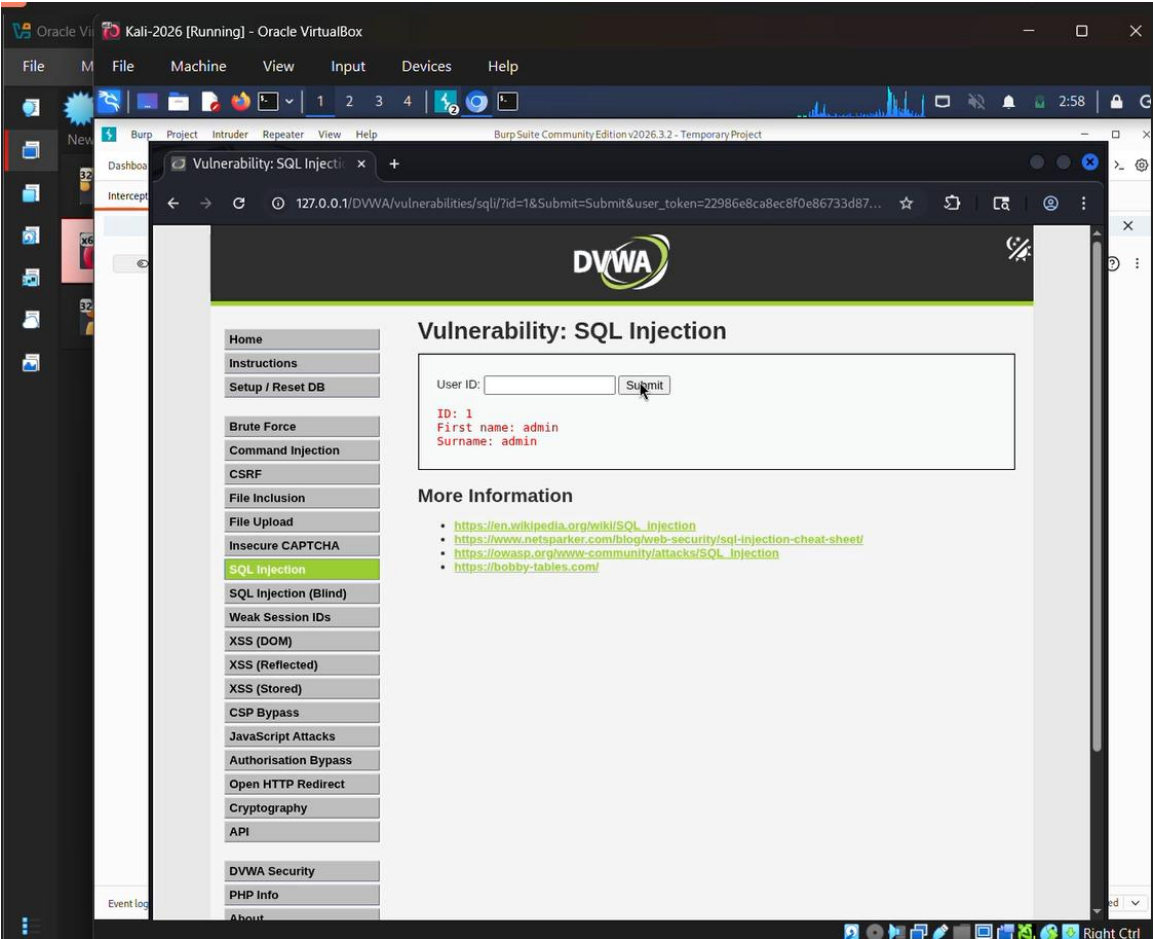
Lab Setup Commands

Command	Description
<code>sudo apt update</code>	Updates the package list from the repositories.
<code>sudo apt upgrade -y</code>	Upgrades all installed packages to their latest versions.
<code>sudo apt install apache2 mariadb-server php php-mysqli php-gd libapache2-mod-php git -y</code>	Installs Apache web server, MariaDB database server, PHP, required PHP modules, and Git.
<code>sudo systemctl start apache2</code>	Starts the Apache web server.
<code>sudo systemctl enable apache2</code>	Enables Apache to start automatically during system boot.
<code>sudo systemctl start mariadb</code>	Starts the MariaDB database service.
<code>sudo systemctl enable mariadb</code>	Enables MariaDB to start automatically during system boot.
<code>cd /var/www/html</code>	Navigates to the Apache web root directory.
<code>sudo rm index.html</code>	Removes the default Apache web page.
<code>sudo git clone https://github.com/digininja/DVWA.git</code>	Downloads the DVWA application from the GitHub repository
<code>sudo chown -R www-data:www-data /var/www/html/DVWA</code>	Changes ownership of the DVWA directory to the Apache web server user.
<code>sudo chmod -R 755 /var/www/html/DVWA</code>	Sets appropriate permissions for the DVWA files and folders.
<code>cd /var/www/html/DVWA/config</code>	Opens the DVWA configuration directory.
<code>sudo cp config.inc.php.dist config.inc.php</code>	Creates the active DVWA configuration file from the sample configuration.
<code>sudo nano config.inc.php</code>	Opens the DVWA configuration file for editing database settings.
<code>sudo mysql</code>	Opens the MariaDB command-line interface.
<code>CREATE DATABASE dvwa;</code>	Creates the DVWA database.
<code>CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';</code>	Creates a new database user for DVWA.
<code>GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';</code>	Grants full permissions on the DVWA database to the created user.
<code>FLUSH PRIVILEGES;</code>	Reloads the privilege tables so changes take effect immediately.
<code>EXIT;</code>	Exits the MariaDB console.
<code>sudo systemctl restart apache2</code>	Restarts Apache after configuration changes.
<code>php -v</code>	Displays the installed PHP version.
<code>apache2 -v</code>	Displays the installed Apache version.
<code>git --version</code>	Displays the installed Git version.
<code>burpsuite</code>	Launches Burp Suite Community Edition.
<code>curl -I http://localhost/DVWA</code>	Displays the HTTP response headers of the DVWA application.
<code>sudo a2enmod headers</code>	Enables the Apache Headers module.
<code>sudo systemctl restart apache2</code>	Restarts Apache after enabling the Headers module.
<code>sudo systemctl status apache2</code>	Displays the current status of the Apache service.
<code>sudo systemctl status mariadb</code>	

	Displays the current status of the MariaDB service.
--	---

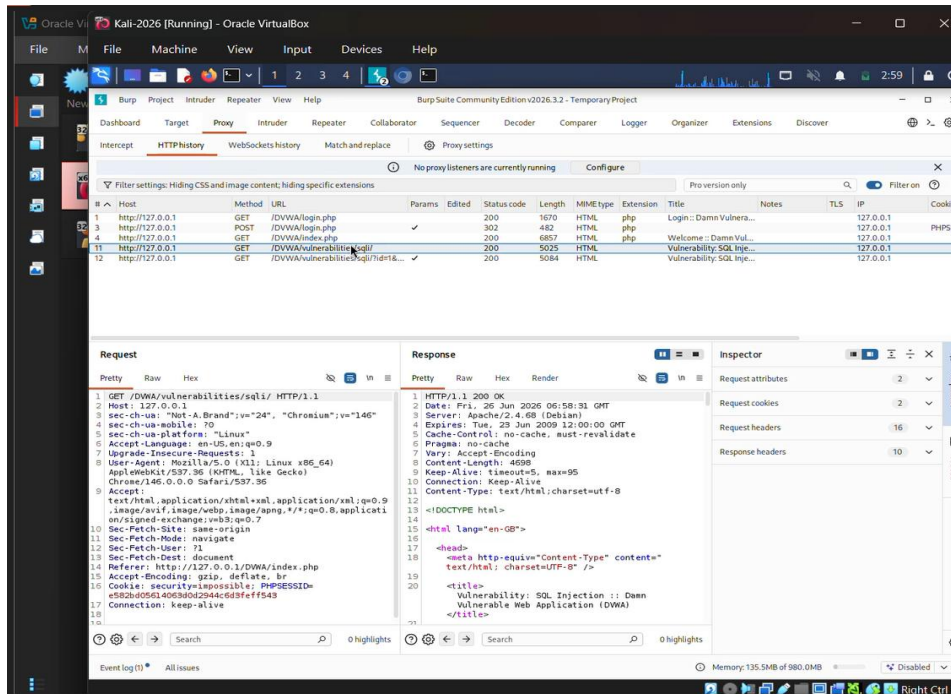
SQL Injection

Payload example: 1' OR '1'='1. Explain impact and mitigation using prepared statements.



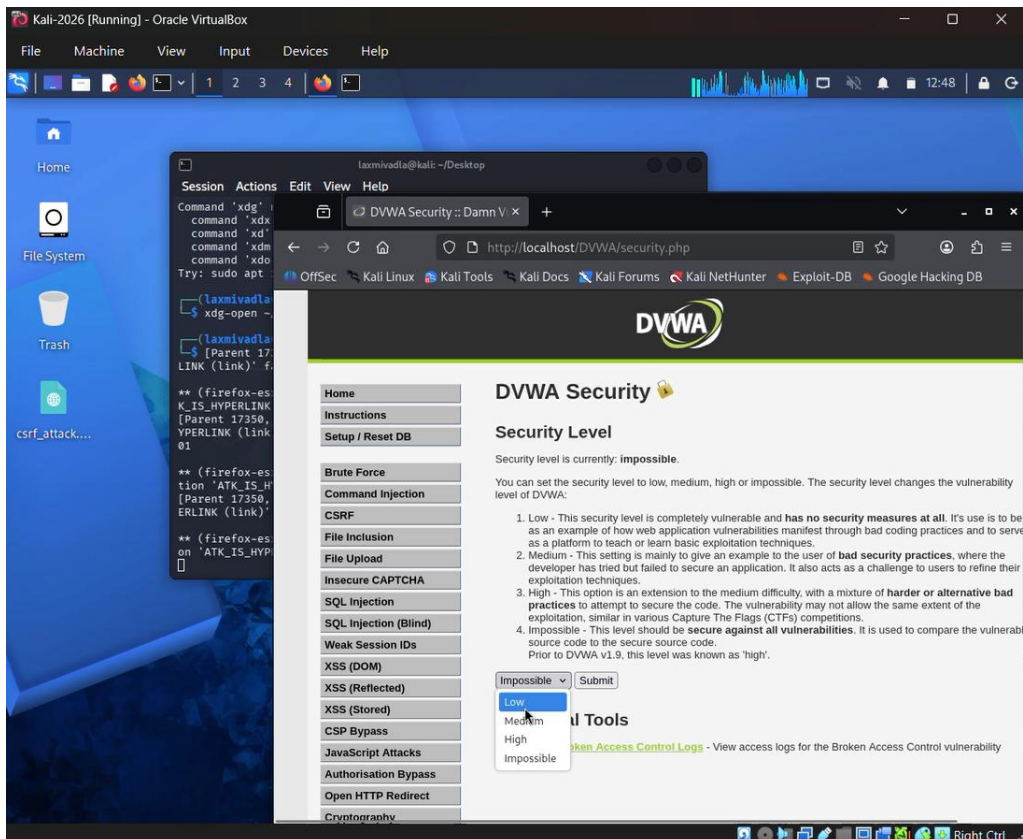
Stored and Reflected XSS

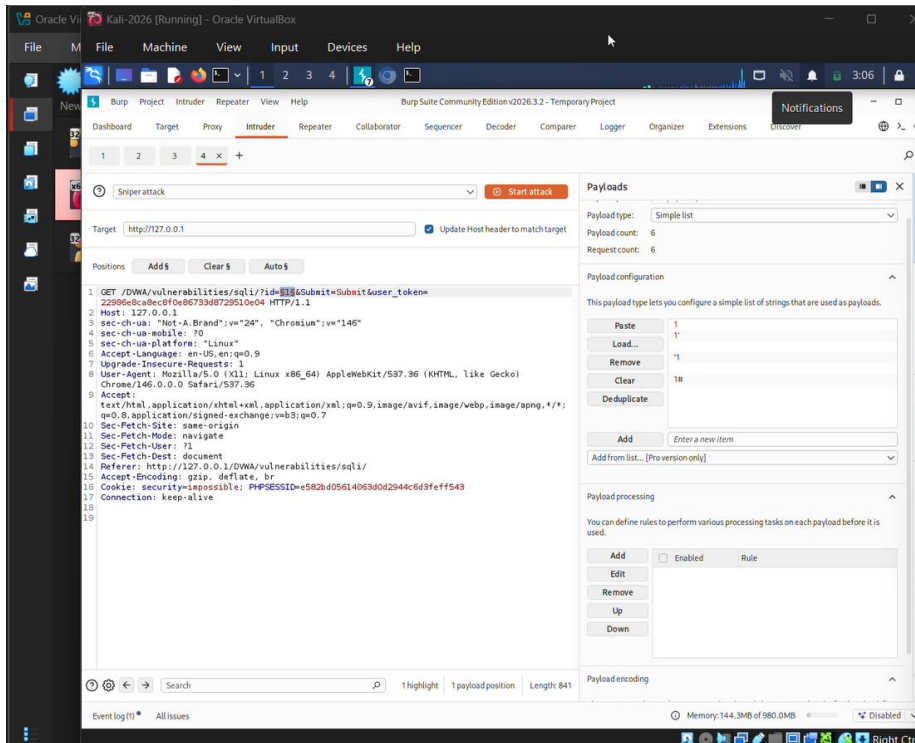
Demonstrate JavaScript execution using safe lab payloads and explain output encoding and CSP.



CSRF

Demonstrate forged request concept and explain CSRF tokens and SameSite cookies.



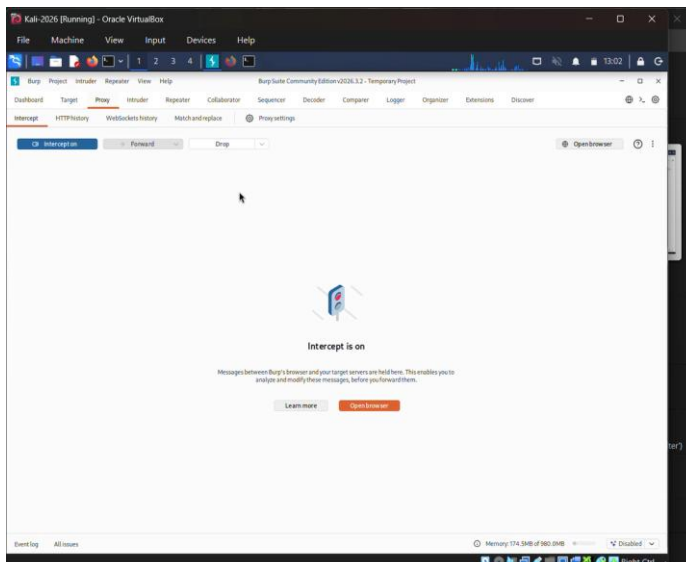


Local File Inclusion

Example payload: ../../../../etc/passwd. Explain path validation and allowlists.

Burp Suite

Intercept requests, modify parameters, use Intruder for fuzzing.

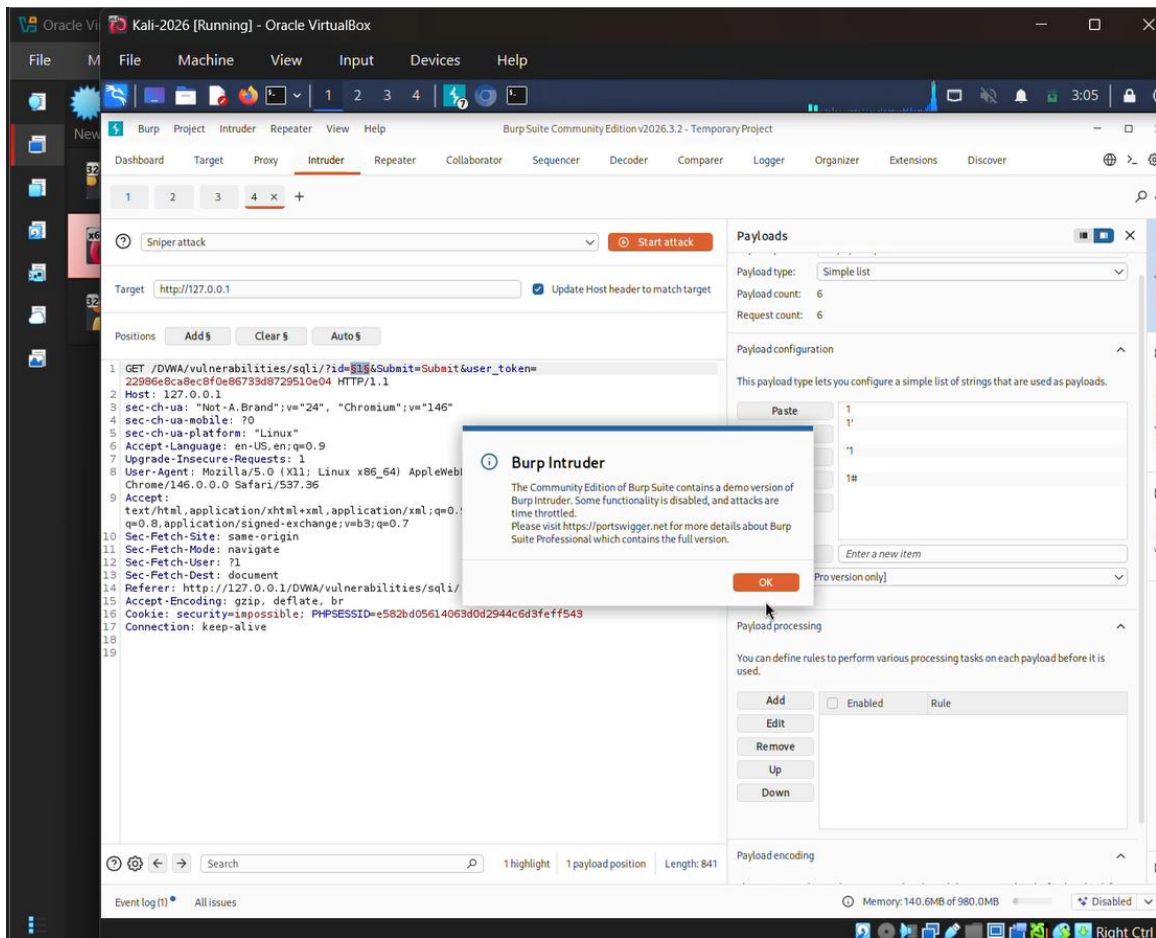


Security Headers

Review and configure CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy.

Results

Successfully demonstrated vulnerabilities in DVWA and documented mitigations.



Conclusion

This project successfully demonstrated the identification and analysis of common web application vulnerabilities using DVWA in a controlled laboratory environment. Practical experiments on **SQL Injection (SQLi)**, **Cross-Site Scripting (Stored and Reflected XSS)**, **Cross-Site Request Forgery (CSRF)**, and **Local File Inclusion (LFI)** provided valuable insight into how insecure web applications can be exploited. **Burp Suite** was used to intercept and analyze HTTP requests, while HTTP Security Headers were examined to understand their role in strengthening web application security.

The project also emphasized the importance of secure coding practices and effective mitigation techniques, including **prepared statements, input validation, output encoding, Content Security Policy (CSP), CSRF protection mechanisms, and secure file handling**. These practices help reduce the risk of attacks and improve the overall security of web applications.

Overall, this task enhanced practical knowledge of ethical hacking, web application penetration testing, and secure software development. The skills gained through this project provide a strong foundation for identifying, preventing, and mitigating web security vulnerabilities in real-world applications while following responsible and ethical cybersecurity practice.

References

The following references were used to understand web application security concepts, configure the testing environment, and perform the practical experiments:

1. **OWASP Foundation.** *OWASP Top 10 – Web Application Security Risks.*
<https://owasp.org/www-project-top-ten/>
2. **DVWA (Damn Vulnerable Web Application) – Official GitHub Repository.**
<https://github.com/digininja/DVWA>
3. **Kali Linux Documentation.**
<https://www.kali.org/docs/>
4. **Apache HTTP Server Documentation.**
<https://httpd.apache.org/docs/>
5. **PHP Official Documentation.**
<https://www.php.net/docs.php>
6. **MariaDB Documentation.**
<https://mariadb.com/docs/>
7. **Burp Suite Community Edition Documentation.**
<https://portswigger.net/burp/documentation>
8. **Oracle VM VirtualBox User Manual.**
<https://www.virtualbox.org/manual/>
9. **Mozilla Developer Network (MDN) Web Docs – Web Security.**
<https://developer.mozilla.org/en-US/docs/Web/Security>
10. **ApexPlanet Cybersecurity Internship – Task 3 Guidelines and Lab Manual.**